



EECS 498/CSE 598: Zero-Knowledge Proofs

Winter 2026

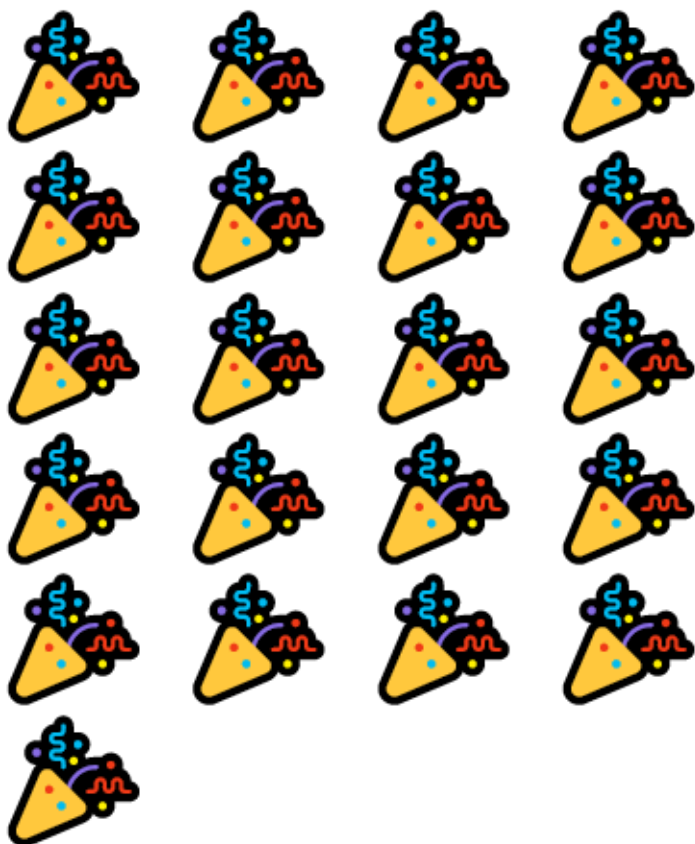
Lecture 24

LAST LECTURE!!!!!!!!!!!!!!

Paul Grubbs

paulgrub@umich.edu

Beyster 4709



Agenda for this lecture

- Announcements
- Background on longfellow-zk case study
- Technical background on longfellow construction
- Overview of construction
- Major technical challenges
- Course retrospective

Agenda for this lecture

- **Announcements**
- Background on longfellow-zk case study
- Technical background on longfellow construction
- Overview of construction
- Major technical challenges
- Course retrospective

Announcements

- P4 and theory pset 2 are out!.
 - Note there will be no P5 nor a final exam. These will be the last assignments of the semester.
- All work for this course must be completed by our final exam date, April 27th
- Reminder: you can get participation credit for going to discussion.

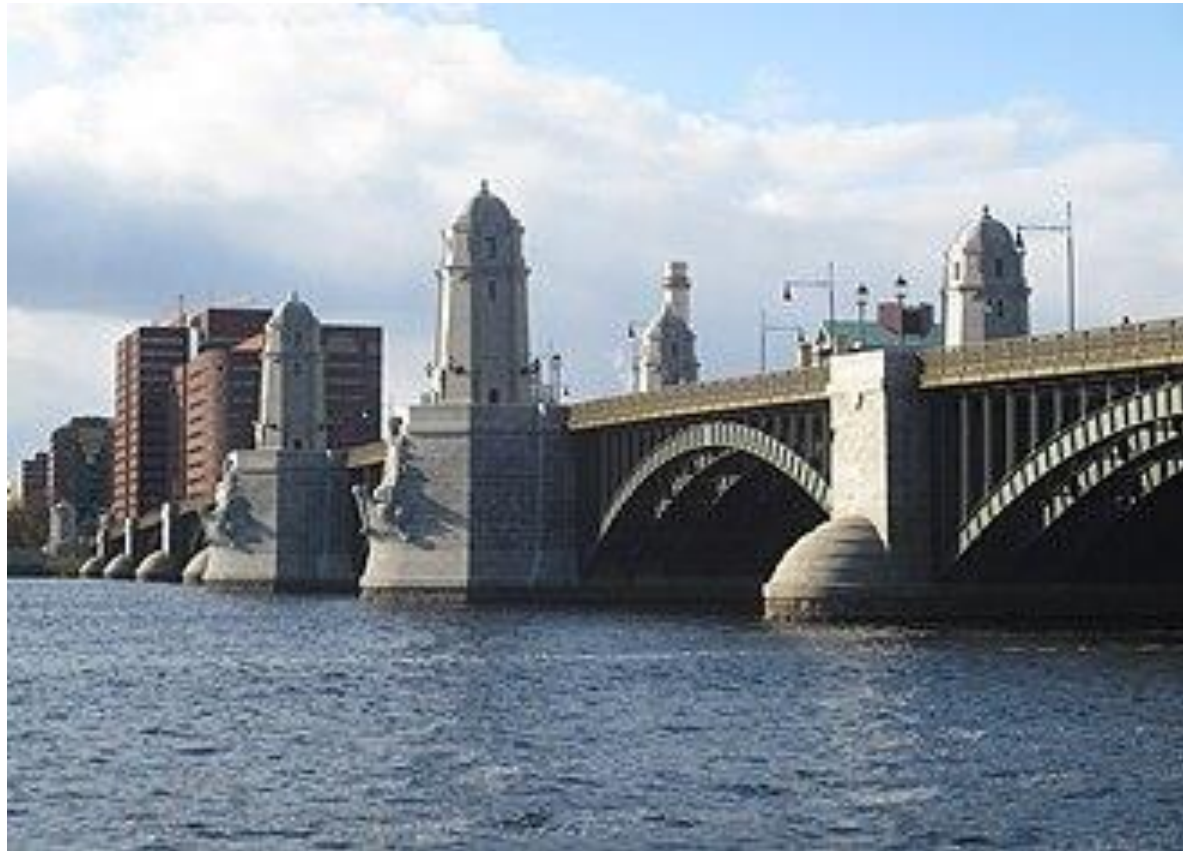


Agenda for this lecture

- Announcements
- Background on longfellow-zk case study
- Technical background on longfellow construction
- Overview of construction
- Major technical challenges
- Course retrospective

Longfellow

Nabeel made a great suggestion last Wednesday: look at Google's ongoing work on deploying zkSNARKs for anonymous credentials. Cryptographic scheme is called "**longfellow**". What is longfellow, exactly?



Longfellow

Anonymous Credentials from ECDSA

Matteo Frigo*
Google

abhi shelat†
Google

December 21, 2024

Abstract

[🏠](#) > [INNOVATION & AI](#) > [TECHNOLOGY](#) > [SAFETY & SECURITY](#)

Opening up ‘Zero-Knowledge Proof’
technology to promote privacy in age
assurance

Jul 03, 2025
1 min read

Code is now available.



Alan Stapelberg
Group Product Manager, Google Wallet

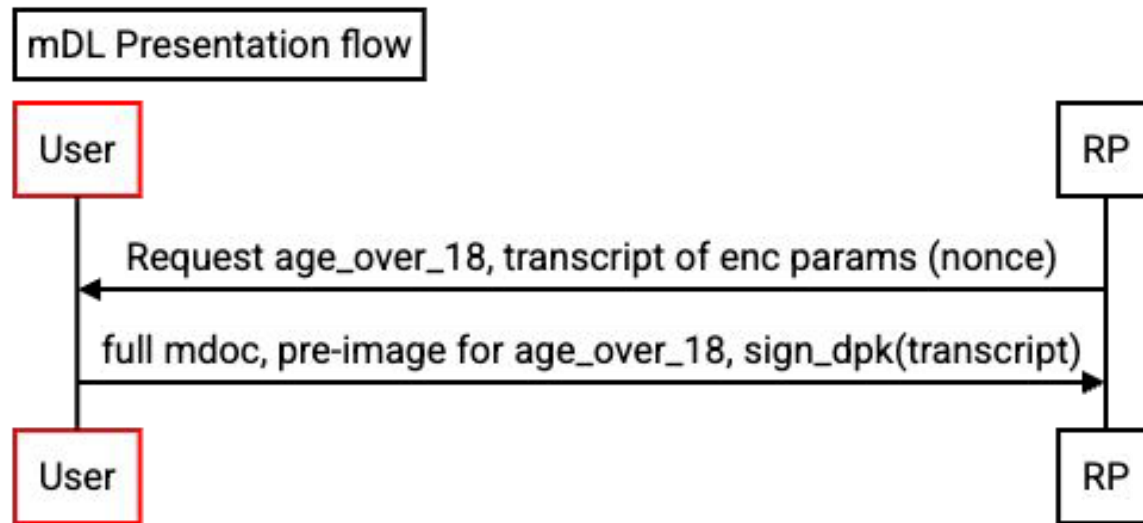
[Share](#)

Longfellow

- Legacy-compatible anonymous credential construction from zkSNARKs.
- Convert government-issued mobile driver's licenses (mDL) into anonymous credential for use on the web. mDLs use "MDOC" format (ISO standard)
- Key application: age verification (prove you're over 18)

Legacy ISO MDOC identity protocol

“The state of Pennsylvania has produced a signature on an mDL document stored on my cell phone that includes the attribute ‘age_over_18 = true’.”



1. Verify Signature of mdoc by California.
2. Parse mdoc to find DPK.
3. Verify Signature of transcript under DPK.
4. Verify pre-image of the “age_over_18” attribute, verify it is set to True.
5. Verify credential expiry condition.

A sample MDOC

```
{ "version": "1.0", "digestAlgorithm": "SHA-256", "docType":  
  "org.iso.18013.5.1.mDL",  
    "valueDigests": { "org.iso.18013.5.1":  
      { 0: h'71CD467C9C1C7A0C889506E97E1335E0FFB81FD19D8D5C848D74D70231ABFA82',  
        1: h'F7D539E2B326A26F53B6BBB9E88FE7DF989075B5E771C8F3A86DDA9851CA377D',  
          ...  
        34: h'4E6DEBDA8C420B79FA8F2780AC555760F63574BBC76C973132627CCA5D5846D7' } },  
    "deviceKeyInfo":  
      { "deviceKey": { 1: 2, -1: 1,  
        -2: h'96313D6C63E24E3372742BFDB1A33BA2C897DCD68AB8C753E4FBD48DCA6B7F9A',  
        -3: h'1FB3269EDD418857DE1B39A4E4A44B92FA484CAA722C228288F01D0C03A2C3D6' } },  
    "validityInfo":  
      { "signed": 0("2023-11-01T09:00:00Z"),  
        "validFrom": 0("2023-11-01T09:00:00Z"),  
        "validUntil": 0("2024-11-15T17:52:30Z") } }
```

← **Salted hash of an attribute in the credential**

← **Device-bound public key.**

Used in the protocol to assert freshness of the presentation.

Sign([Com(attr1), Com(attr2), ..., Com(attrk), <expiration>, <metadata>)

Attributes are encoded using a salted hash to support selective disclosure. E.g., the `age_over_18` attribute:

SHA-256 (`

D818584FA4686469676573744944156672616E646F6D50AC50F651BC04 salt

F001D02163847C9F1CC971656C656D656E744964656E7469666965726B

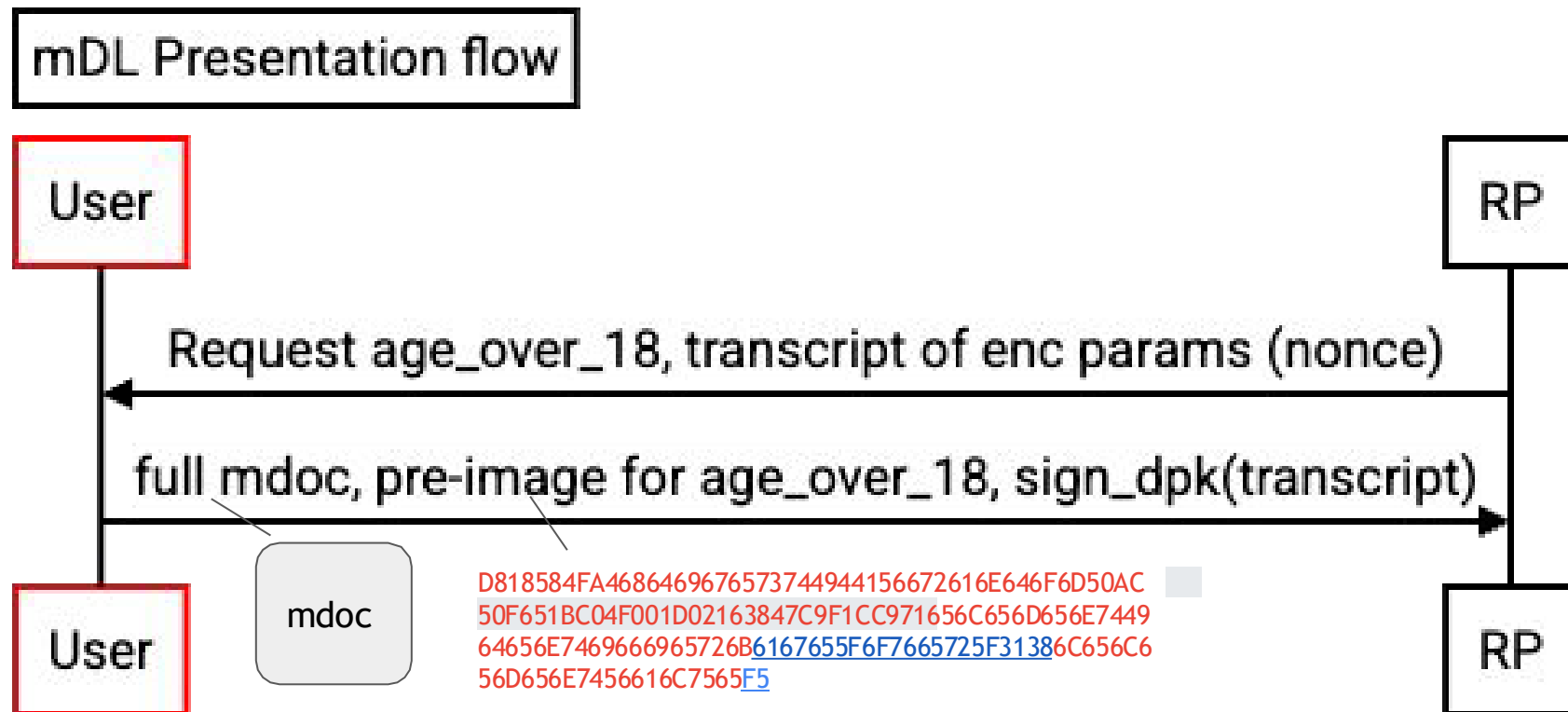
6167655F6F7665725F31386C656C656D656E7456616C7565F5) =

a g e _ o v e r _ 1 8 T

71CD467C9C1C7A0C889506E97E1335E0FFB81FD19D8D5C848D74D70231ABFA82

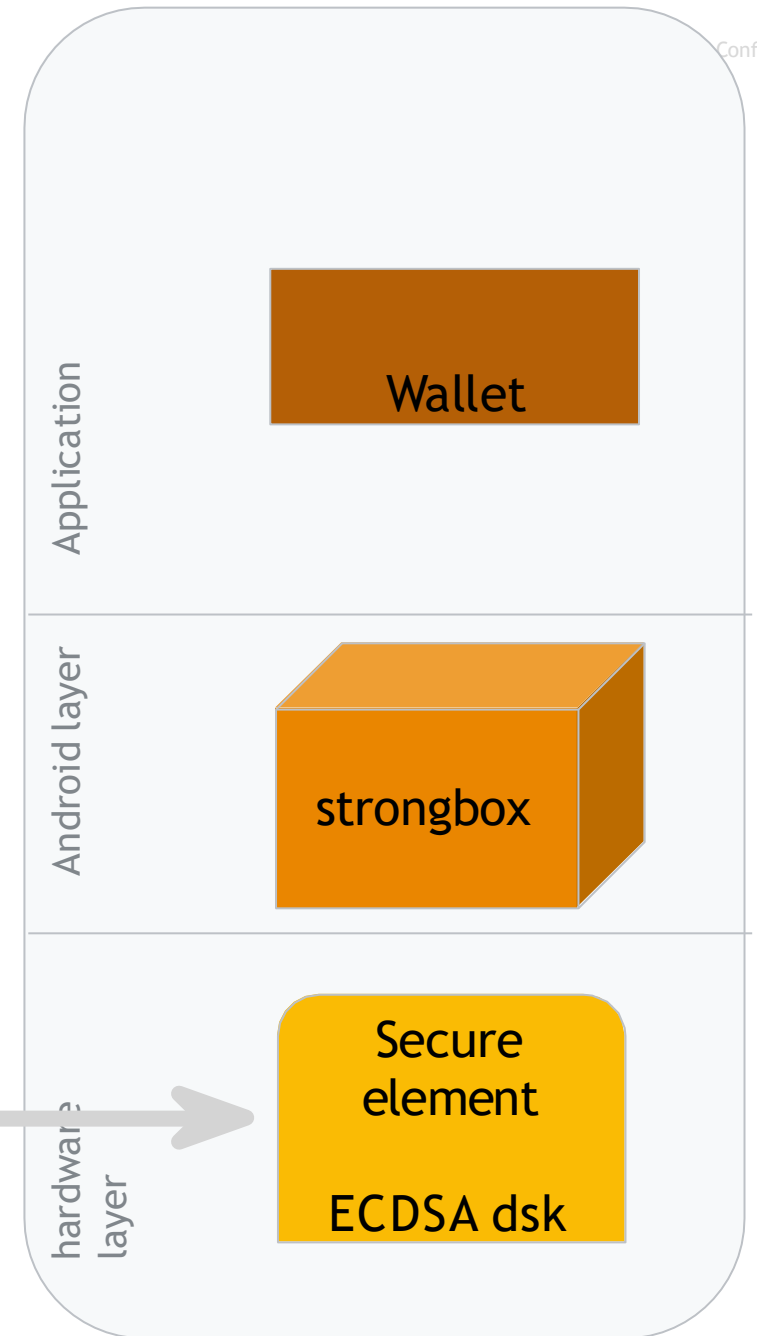
Legacy ISO MDOC identity protocol

“The state of Pennsylvania has produced a signature on an mDL document stored on my cell phone that includes the attribute ‘age_over_18 = true’.”



To prevent users from **sharing credentials**, issuers only create credentials that include a **device bound** public key that is stored here

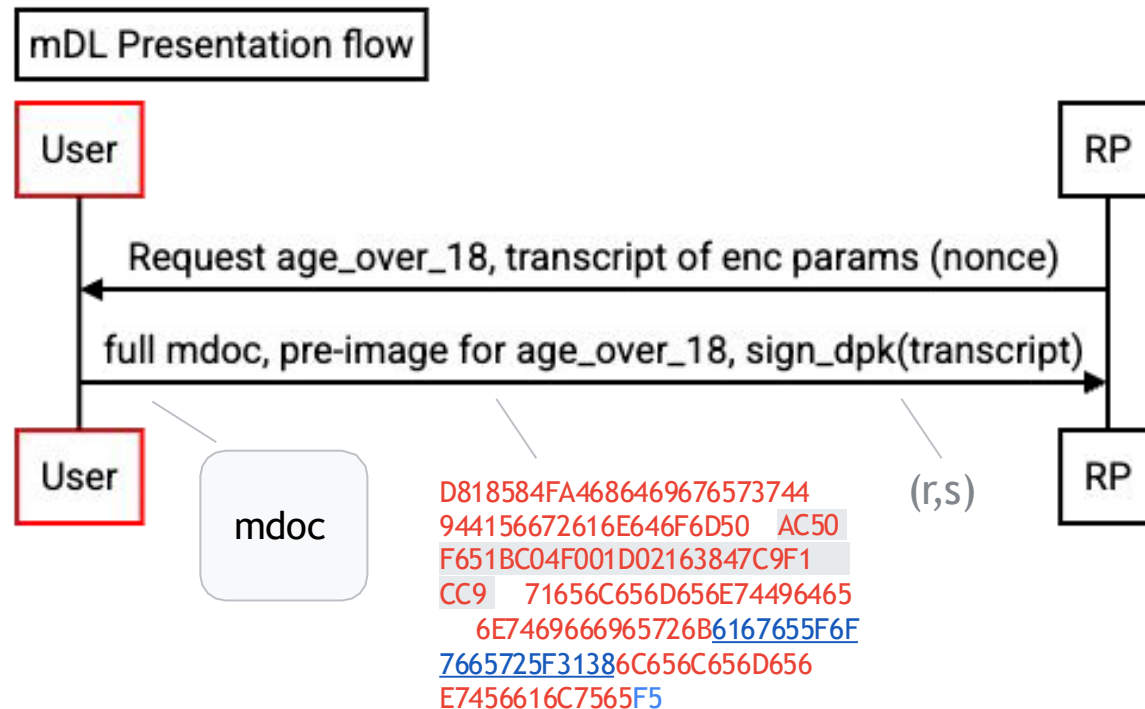
This device-bound key is required in the presentation protocol (sign a transcript with DPK)



Today's Mdoc presentation

Proprietary + Confidential

“The state of Pennsylvania has produced a signature on an mDL document stored on my cell phone that includes the attribute ‘age_over_18 = true’.”



1. Verify Signature of mdoc by California.
2. Parse mdoc to find DPK.
3. Verify Signature of transcript under DPK.
4. Verify pre-image of the “age_over_18” attribute, verify it is set to True.
5. Verify credential expiry condition.

Agenda for this lecture

- Announcements
- Background on longfellow-zk case study
- **Technical background on longfellow construction**
- Overview of construction
- Major technical challenges
- Course retrospective

Technical background

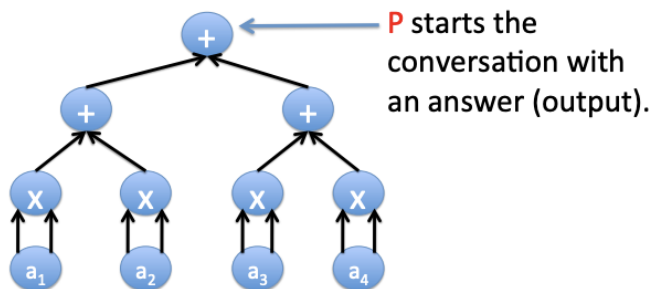


Figure 4.8: Start of GKR Protocol.

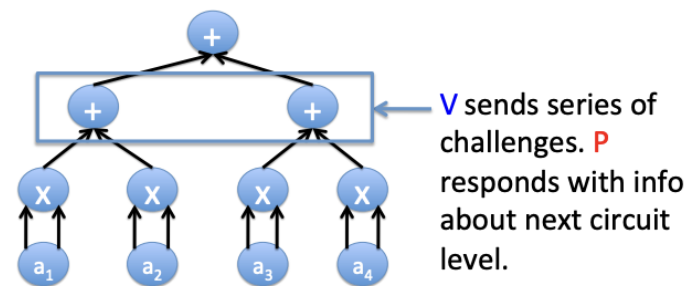


Figure 4.9: Iteration 1 reduces a claim about the output of $\mathcal{C}$ to one about the MLE of the gate values in the previous layer.

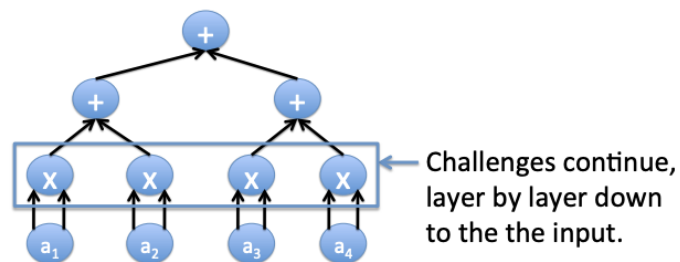


Figure 4.10: In general, iteration i reduces a claim about the MLE of gate values at layer i , to a claim about the MLE of gate values at layer $i + 1$.

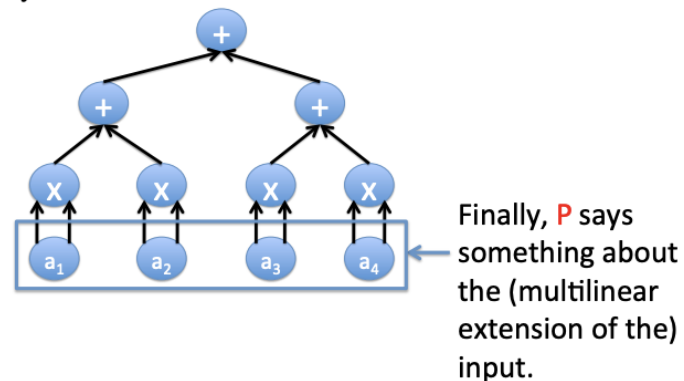


Figure 4.11: In the final iteration, $\mathcal{P}$ makes a claim about the MLE of the input (here, the input of length n with entries in $\mathbb{F}$ is interpreted as a function mapping $\{0, 1\}^{\log_2 n} \rightarrow \mathbb{F}$. Any such function has a unique MLE by Fact 3.5). $\mathcal{V}$ can check this claim without help, since $\mathcal{V}$ sees the input explicitly.

Agenda for this lecture

- Announcements
- Background on longfellow-zk case study
- Technical background on longfellow construction
- **Overview of construction**
- Major technical challenges
- Course retrospective

The Longfellow construction

2nd oldest recipe for ZK [BGGHKMR88]

Run an **IP** $\rightarrow$ Transcript.

Commit to Transcript $\rightarrow$ Com.

ZK for “Com contains T and $\text{IP-verifier}(T)=1$ ”

It just remains to pick, IP, Commit, ZK.

Our Choices

IP: Layered Sumcheck [GKR] (Prover can run in $O(C)$ time!)

Commit/ZK: Ligerio [AHIV] (Only uses SHA256. Simple, fast.)

P convinces V that “there exists w such that $f(\vec{x}, \vec{w}) = 0$ ”

Prover($f, \vec{x}, \vec{w}$)

Verifier($f, \vec{x}$)

Pick a random pad.

Ligero-Commit(w, pad)

com

Run Sumcheck-P(C, x, w)

When Sumcheck sends message
 m_i , send $m_i + \text{pad}_i$.
Record transcript T .

Run Sumcheck-V(C, x)

Send random challenges.
Record transcript T

...

Run Ligero-Prover($\text{com}, \text{Enc-Sumcheck-Verify}(T, x, \text{pad})$)

Ligero-Verify($\text{com},$
 $\text{Enc-Sumcheck-Verify}, T, x, \text{proof}$)

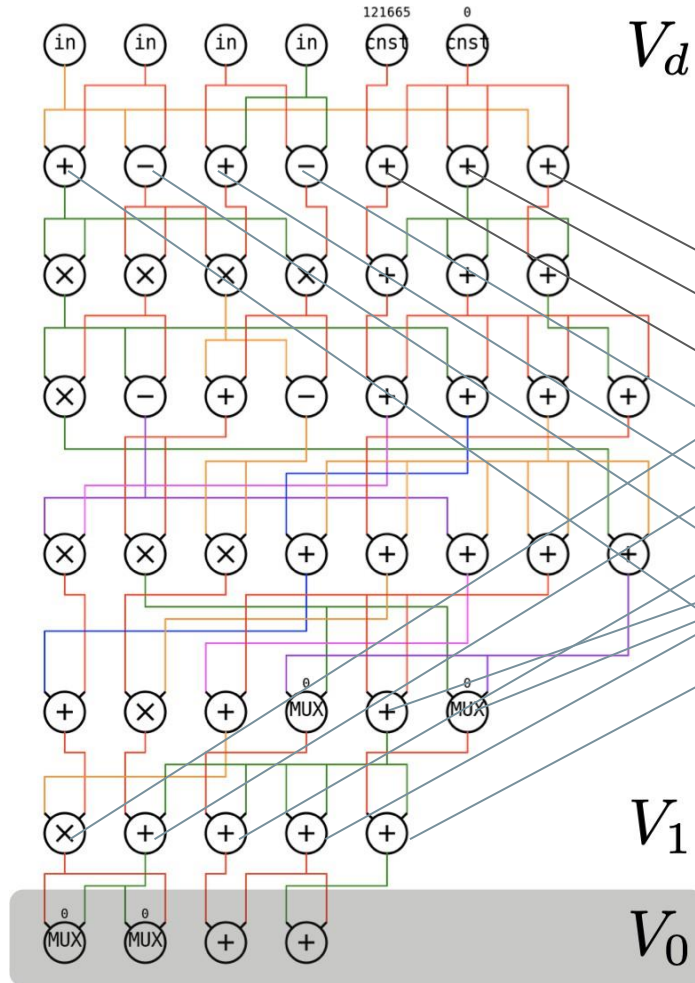


The scheme is inspired by Hyrax [WTSTW17].

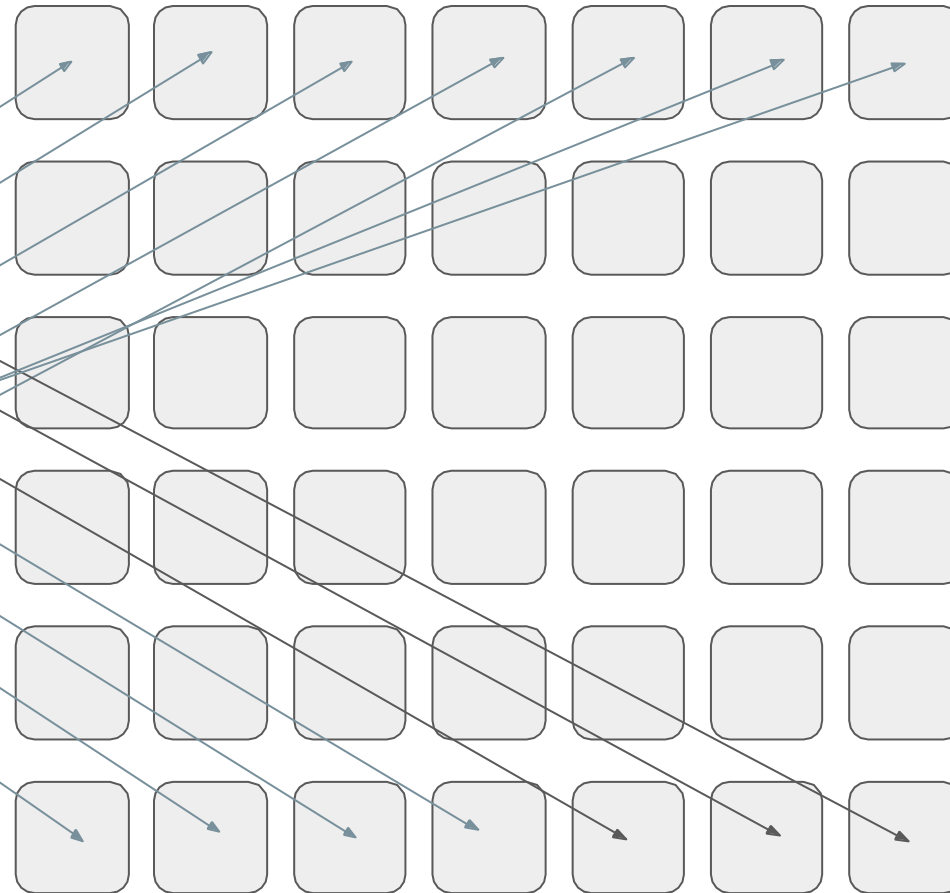
Most TARKs commit to every intermediate wire

E.g., Ligerio, Starkware, ...

Circuit(x,w)



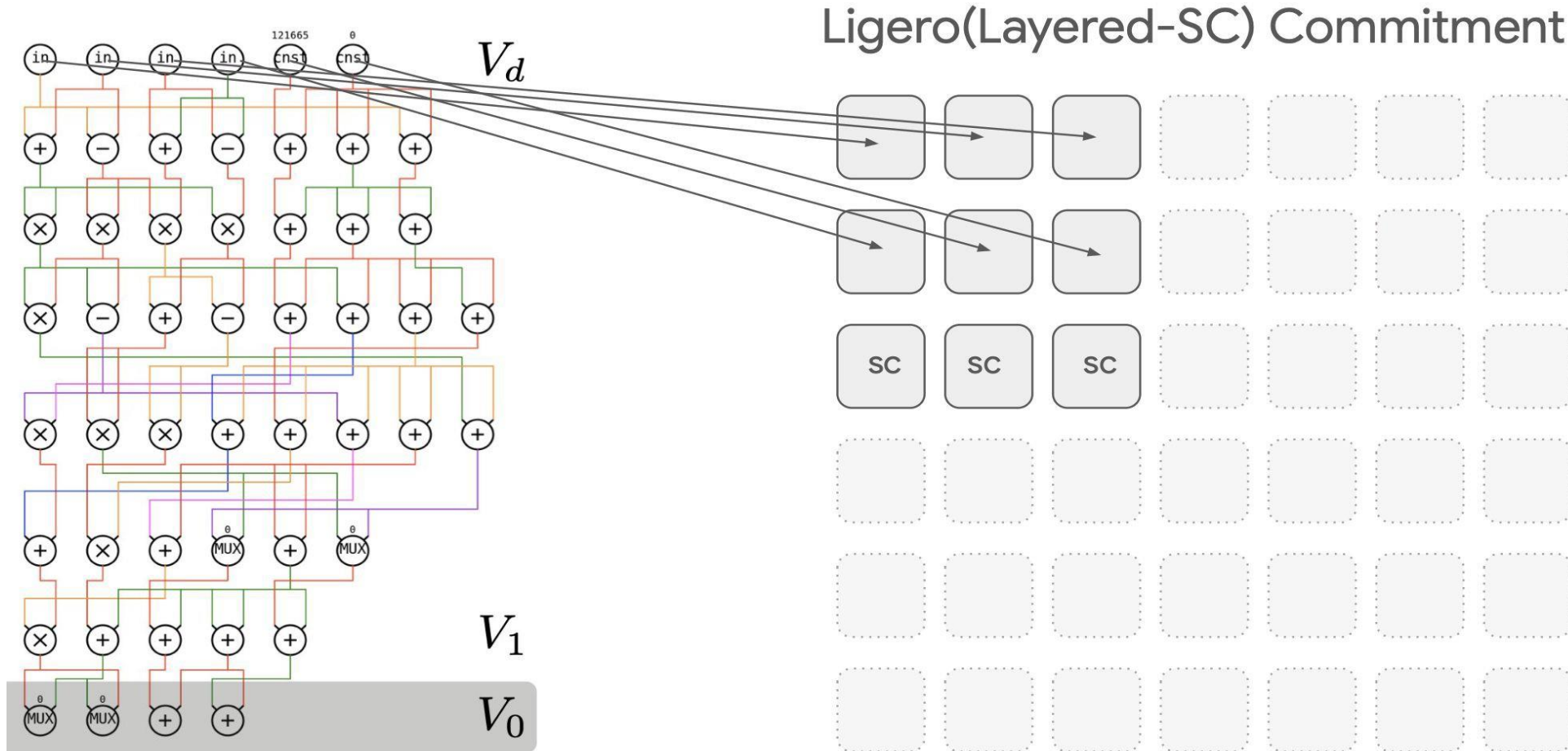
Commitment



The commitment is the expensive part.

Our system commits to input+sc proof

and only verifies the sumcheck proof



Agenda for this lecture

- Announcements
- Background on longfellow-zk case study
- Technical background on longfellow construction
- Overview of construction
- **Major technical challenges**
- Course retrospective

Challenges

Verifying an ECDSA signature

Algorithm 2 Verification Circuit $V(Q, H(m), r, s, r_y)$

- 1: Derive integer e from $H(m)$.
 - 2: Verify that $r, s \in [1, q - 1]$
 - 3: Verify that $Q, R = (r, r_y) \in E$ and $R \neq id$.
 - 4: Verify $id = G \cdot e + Q \cdot r - R \cdot s$
-

Prior work (eg circom-ECDSA, Stark, Plonk,...) needs to emulate the field math of F_p , F_q .

ECDSA verification is a small circuit in F_P256

Table 1: Circuit size and depth for ECDSA verification.

	DEPTH	QUADS	TERMS	INPUTS
Multi-exponentiation	7	19,534	37,550	1,038
Range check + rest	12	5,475	10,569	1,038
Total	12	23,453	47,598	1,038

~50x fewer wires
to commit

Size of input vs size of circuit = 50x.

Finite field used by our circuit is over the base field of the P256 curve. Prior work was expensive b/c it simulated math in F_p in an NTT-friendly F_q .

ZK Statement to prove for ISO MDOC identity

Given the public values (PK_II, “age_over_18”, transcript, time_now), there exists

a 2231 byte string MDL, a hash e_1, a hash h_2, an index X, a signature sig_1, ..., a 32-b nonce, a pk DPK,

a pair of strings time_start, time_end, and a signature sig_2 such that:

```
e_1 = SHA-256(MDL) and  
p256.verify( sig_1, e_1, PK_II ) = true and
```

← Costly part

```
h_2 = MDL[valueDigests][org.iso.18013.5.1]['age_over_18']] and  
h_2 = SHA-256(nonce, 'age_over_18', 'True') and
```

```
DPK = MDL[deviceKeyInfo][deviceKey][-2, -3] and  
p256.verify( sig_2, transcript, DPK ) = true and
```

Device
binding

```
time_start = MDL[validityInfo][validFrom] and  
time_end = MDL[validityInfo][validUntil] and  
time_start < time_now < time_end and
```

~ 7.2m “wires”,
3m “quads”,
600k in Fp,
6.7m in 2^{128}

Agenda for this lecture

- Announcements
- Background on longfellow-zk case study
- Technical background on longfellow construction
- Overview of construction
- Major technical challenges
- **Course retrospective**

Course retrospective

We've come a long way, and covered a lot of material!!

Part 1: Foundations

Part 2: Backends

Part 3: Frontends

Part 4: Advanced topics and applications

Thank you for a great semester!!!

Special thanks to our GSI Chad!